Comprendre le positionnement des objets dans Flet

Cours d'introduction avant le code - construire le bon modèle mental

L’idée fondamentale

Dans Flet, un contrôle ne décide généralement pas tout seul où il se trouve. Son parent détermine le contexte de placement. Une fois cette idée comprise, Row, Column, Stack, alignment, padding et margin deviennent beaucoup plus faciles à distinguer.

Objectif du cours

Comprendre comment Flet organise l’espace, avant de chercher à reproduire visuellement un écran d’iPod Touch. Le but n’est pas encore de produire une interface jolie : il s’agit de savoir qui place quoi, par rapport à quoi, et selon quelle règle.

1. Flet fonctionne avec une hiérarchie

Une interface Flet peut être vue comme un arbre de contrôles :

```
PAGE
└── CONTENEUR
    ├── TEXTE
    ├── BOUTON
    └── IMAGE
```

Ou, par exemple :

```
PAGE
└── ROW
    ├── BOUTON
    ├── BOUTON
    └── BOUTON
```

Ou encore :

```
PAGE
└── STACK
    ├── IMAGE
    ├── TEXTE
    └── BOUTON
```

La notion importante est la relation parent → enfants. Le parent fournit les règles de disposition, d’espace ou de superposition utilisées par ses enfants.

2. Pourquoi page.add() ne suffit pas

Dans le fichier de départ, les contrôles sont ajoutés directement à la page avec plusieurs appels à **page.add()**. Cela crée une structure simple où les éléments suivent le système de mise en page de la vue racine, plutôt qu’un système de coordonnées libres. La documentation Flet décrit

d’ailleurs la View comme se comportant comme une Column pour son layout.

```
page.add(Text(...))
page.add(Text(...))
page.add(Button(...))
page.add(Button(...))
```

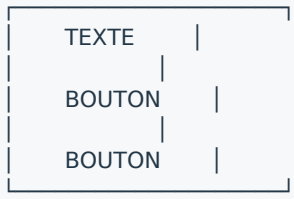
Mentalement, il faut donc cesser de voir **page.add()** comme « mets cet objet à tel endroit ». C’est plutôt « ajoute cet objet à cette structure de mise en page ».

3. Deux grandes philosophies de placement

Organiser	Positionner
Tu donnes une règle générale : les éléments vont en ligne, en colonne, en grille, etc. Exemple : Row, Column.	Tu disposes un élément dans une zone selon une position précise. Stack avec left/top/right/bottom répond à ce besoin.

4. Column : organiser verticalement

Column peut être imaginée comme une étagère verticale. Elle prend une collection d’enfants et les dispose verticalement.



Ici, on ne donne pas à chaque bouton une coordonnée X/Y. On dit simplement : « voici mes éléments, organise-les verticalement ».

5. Row : organiser horizontalement

Row suit la même logique, mais dans l’axe horizontal. C’est donc un outil naturel pour les quatre boutons du bas de ton homescreen.



La propriété alignment permet ensuite de contrôler la manière dont l’ensemble est réparti dans l’espace disponible : au début, au centre, autour, entre les éléments, etc. Le principe reste organisationnel, pas individuel.

6. Stack : l’espace de superposition

Stack change de philosophie. Il permet d’empiler des contrôles les uns au-dessus des autres et constitue donc un bon point de départ lorsqu’on veut une scène dans laquelle les éléments peuvent occuper des positions précises.



On peut alors raisonner avec des distances comme left et top, ou right et bottom. La documentation de Flet décrit ces propriétés comme du positionnement absolu lorsqu'un contrôle est descendant d'un Stack ou d'une zone d'overlay.

7. Les coordonnées sont relatives au parent

C'est une subtilité fondamentale. left=100 ne signifie pas « 100 pixels depuis le bord de l'écran ». Il faut comprendre « 100 unités depuis le bord gauche de la zone de référence fournie par le parent ».

```
PAGE
└── STACK A
    └── BOUTON
        left = 100

Le bouton se mesure dans le contexte de STACK A.
```

Si un autre Stack devient le parent, le même left=100 change de référence. Le positionnement dépend donc toujours de la hiérarchie.

8. Container : la boîte de l'interface

Container est à comprendre comme une boîte qui peut servir à donner une taille, de la décoration, du padding, du margin, de l'alignement et à contenir un autre contrôle. Il est donc utile de distinguer la position de la boîte et la position du contenu dans la boîte.



La boîte peut être placée dans son parent. Ensuite, son contenu peut être aligné à l'intérieur de cette boîte. Ce sont deux niveaux différents.

9. Positionnement, alignment, padding, margin : ne pas les confondre

Notion	Question à se poser
Positionnement	Où est la boîte dans son espace parent ?
Alignment	Comment le contenu est-il placé dans l'espace qui lui est attribué ?
Padding	Quel espace existe à l'intérieur de la boîte entre son bord et son contenu ?
Margin	Quelle distance existe autour de la boîte par rapport aux autres éléments ?
Width / Height	Quelle est la taille disponible ou imposée à la boîte ?

10. Une analogie simple : construire le boîtier de l'iPod

Imagine que tu fabriques physiquement ton futur lecteur. Tu as un boîtier, puis un écran, puis des éléments posés dans l'écran. Chaque niveau fournit un nouvel espace de référence.

```
BOÎTIER
└── ÉCRAN
    └── INTERFACE
        ├── HEURE
        ├── APPLICATIONS
        └── DOCK
```

Cette analogie « boîte dans boîte » est utile pour Flet : les coordonnées et règles prennent leur sens à l'intérieur du parent qui les contient.

11. Combiner les systèmes : c'est là que l'interface devient puissante

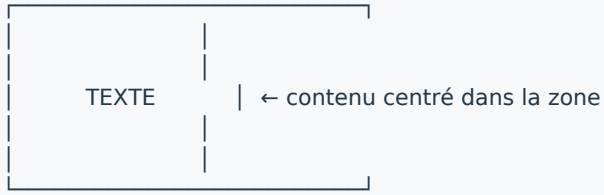
Tu n'as pas à choisir une seule technique pour toute l'interface. Tu peux utiliser un Stack pour la grande composition, puis Row ou Column à l'intérieur de certaines zones.

```
PAGE
└── STACK
    ├── Status Bar
    │   └── Row
    ├── Heure
    ├── Applications
    │   └── Row / Column / Grid
    └── Dock
        └── Row
```

C'est probablement le modèle le plus intéressant pour ton homescreen iPod : Stack pour la scène générale, puis des conteneurs spécialisés pour organiser les groupes d'éléments.

12. alignment : penser « contenu dans un espace »

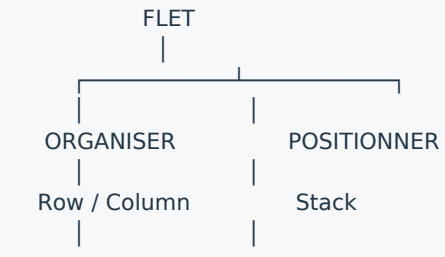
Il faut éviter de confondre alignment avec « coordonnées ». Une façon plus juste de le voir est : comment placer le contenu à l'intérieur de la zone dont il dispose ? Par exemple, un texte peut être placé vers le haut, au centre ou vers le bas dans son conteneur.

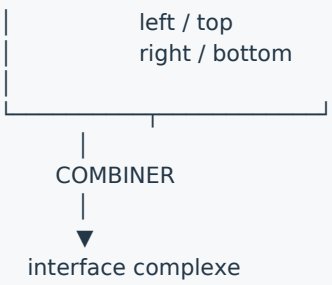


13. expand : à mettre de côté pour le moment

expand devient très utile lorsque tu veux qu'un contrôle prenne une partie ou tout l'espace disponible de son parent. Mais pour ton premier prototype spatial, ce n'est pas la notion prioritaire. L'essentiel est d'abord de comprendre parent, organisation, positionnement et contenu.

14. La carte mentale à retenir





Container sert ensuite à créer et contrôler les boîtes autour du contenu : taille, décoration, padding, margin et alignement. Voilà la structure mentale qui permet ensuite de lire la documentation Flet sans se perdre dans toutes les propriétés.

15. Application à ton homescreen iPod

```
STACK = le canevas principal
│
├── Zone haute = statut / heure / batterie
├── Zone centrale = applications
└── Zone basse = dock

Puis, à l'intérieur :
- Row pour une ligne d'éléments
- Column pour une organisation verticale
- éventuellement une grille pour les icônes
```

L'ordre conseillé est donc : 1) comprendre le canevas, 2) placer les grandes zones, 3) organiser les groupes d'éléments, 4) seulement ensuite travailler le design.

16. La phrase à garder en tête

La règle mentale

Quand tu rencontres une propriété de layout dans Flet, demande-toi : « Est-ce que cette propriété dit où placer l'objet, comment organiser plusieurs objets, ou comment placer le contenu à l'intérieur d'un objet ? »

Cette question suffit à distinguer une grande partie des outils de layout et évite de traiter Flet comme un simple écran rempli de coordonnées.

Note pour les archives

Cours préparatoire réalisé à partir du prototype Flet fourni et de la démarche de conception du projet iPod maison. Le document est volontairement centré sur le modèle mental du layout et laisse le code détaillé pour l'étape suivante.